INF384 · Temas Avanzados en Ingeniería de Software 1

Laboratorio 2 - Trabajo Previo

Trabajo previo

Este trabajo previo no tiene nota. Es necesario para que puedan trabajar en el laboratorio.

Los bloques A y B se pueden hacer en cualquier orden. **El bloque C tiene que estar terminado antes de llegar al laboratorio**.

Bloque A — Sintaxis de GitHub Actions

En el Laboratorio 1 ejecutaron un workflow siguiendo un instructivo. En este laboratorio van a modificar uno. Este bloque es para que lleguen entendiendo el workflow.

Qué revisar

Estructura del archivo

Un workflow vive en `.github/workflows/` y es un archivo `.yml`. Tiene tres partes: cuándo se dispara (`on`), qué trabajos ejecuta (`jobs`), y qué pasos tiene cada trabajo (`steps`).

`on` — los triggers

Define los eventos que disparan el workflow. Los mas comunes:

- `push` — cuando se sube código a una rama
- `pull_request` — cuando se abre o actualiza un pull request
- `workflow_dispatch` — ejecución manual desde la pestaña Actions

Cada uno acepta filtros por rama. Revisen cómo se escribe `branches:` debajo de un trigger y qué significa que un evento tenga filtro y otro no.

`jobs` y `runs-on`

Cada job corre en una máquina virtual limpia. `runs-on` define el sistema operativo. Dos jobs del mismo workflow **no comparten archivos** salvo que se lo indiquen explícitamente.

`steps` — `uses` vs `run`

- `uses` invoca una action publicada por alguien más. Se escribe `owner/nombre@version`.

- `run` ejecuta un comando en la máquina virtual, como si corrieras un bash o cmd en tu máquina, pero se ejecuta en el pipeline en el entorno de la VM que está usando el workflow.

`with`

Los parámetros que se le pasan a una action invocada con `uses`. Van indentados debajo.

`needs`

Declara que un job depende de otro. Si el job del que depende falla, el que declara `needs` no se ejecuta. Presten atención a esto.

`${{ secrets.NOMBRE }}`

Referencia a un secreto guardado en el repositorio. El valor nunca aparece en el archivo ni en los logs. Es lo que usamos para el token de Vercel.

Leer una ejecución

En la pestaña Actions de su repositorio: cada ejecución muestra sus jobs, cada job sus steps, y cada step su log. Cuando algo falla, el step que falló queda marcado. Aprendan a llegar rápido al log del step que falló, porque lo van a hacer varias veces durante la sesión.

Ejercicio A

Abran el archivo de workflow de su repositorio y respondan las siguientes. Son preguntas de lectura del archivo, no es necesario que modifiquen nada en el workflow.

1. ¿Cuántos jobs tiene el workflow y cómo se llaman?
2. ¿Qué eventos disparan el workflow?
3. Nombra un paso que use `uses` y uno que use `run`. ¿Cuál es la diferencia entre los dos?
4. ¿Qué versión de Python se declara y en qué paso?
5. ¿Qué hace `fetch-depth: 0` en el paso de checkout?
6. ¿Qué comando genera el reporte de cobertura y en qué archivo lo deja?
7. ¿Cómo llega el token de SonarCloud al paso de análisis? ¿Aparece su valor escrito en el archivo?

Si el paso de pruebas falla, ¿qué pasa con el paso de análisis que viene después, dentro del mismo job?

Bloque B — SonarCloud: interfaz y métricas

En la Sesión de clase 3 vimos el concepto de quality gate y la diferencia entre medir sobre todo el código y medir sobre el código nuevo. Este bloque es para que vean eso en pantalla antes de tener que configurarlo.

Qué revisar

Quality gate

Es el conjunto de condiciones que el análisis tiene que cumplir. El resultado es binario: **Passed** o **Failed**. En un proyecto, el resultado aparece arriba, en el resumen.

Revisen qué condiciones trae el quality gate por defecto y sobre qué se evalúa cada una.

New code VS Overall code

En el resumen de un proyecto hay dos pestañas: **New Code** y **Overall Code**. Miren este proyecto público: https://sonarcloud.io/summary/new_code?id=aws_aws-sdk-java-v2&branch=master (Es el proyecto de SDK de AWS), vean como cambian las métricas cuando comparan New Code y Overall Code.

El quality gate por defecto evalúa sobre New Code, no sobre Overall Code. Entiendan por qué antes de llegar a la sesión.

Las métricas

- **Coverage** — porcentaje de líneas ejecutadas por las pruebas. Sale de un reporte que produce el test runner, no lo calcula SonarCloud.
- **Duplications** — porcentaje de líneas duplicadas.
- **Issues** — hallazgos, clasificados por severidad.
- **Security Hotspots** — código que requiere revisión humana. **No es lo mismo que un issue**: un issue es un problema; un hotspot es algo que hay que mirar y decidir.
- **Technical Debt** — estimación de tiempo para corregir todos los issues.

Del mismo proyecto de AWS SDK en SonarCloud (https://sonarcloud.io/summary/new_code?id=aws_aws-sdk-java-v2&branch=master) revisen las métricas y anoten los valores de cada una.

Bloque C — Habilitación del entorno

C.1 — Repositorio del grupo

1. Hagan **fork** del repositorio del curso https://github.com/jdelcastillos-pucp/INF384-lab2-base
2. En la pantalla de fork, cambien el nombre a `INF384-lab2-<código-PUCP>`.
3. El fork es público porque el original lo es. No cambien la visibilidad: el plan gratuito de SonarCloud no analiza repositorios privados.
4. Entren a la pestaña **Actions** del fork y confirmen la activación de los workflows. **Hasta que no hagan esto, el workflow no se puede ejecutar y el paso C.5 no funciona.**

C.2 — Cuenta y organización en SonarCloud

1. Entren a sonarcloud.io y creen la cuenta con **Log in with GitHub**.
2. Creen una organización en el plan **Free**. Vincúlenla a la cuenta de GitHub donde está el repositorio.
3. Importen el repositorio del grupo como proyecto.

C.3 — Desactivar el análisis automático

En el proyecto: **Administration** → **Analysis Method** y desactiven **Automatic Analysis**.

Este paso no es opcional. Si queda activo, SonarCloud analiza el código por su cuenta, por fuera del workflow, y el proyecto va a aparecer verde sin que el pipeline haya hecho nada. Van a creer que funciona.

C.4 — Token y secreto

1. En SonarCloud generen un token de análisis: **My Account** → **Security**.

El token se muestra una sola vez. Cópienlo antes de cerrar.

2. En el repositorio de GitHub, vayan a **Settings** → **Secrets and variables** → **Actions** → **New repository secret**.
3. Nombre: `SONAR_TOKEN`. Valor: el token.
4. Ingresen a https://sonarcloud.io/account y anoten la **organization key que figura encima de "User"**, en el ejemplo es "jdelcastillos-pucp". copien la suya propia para que la puedan configurar en su repositorio como variable de entorno.



5. En el repositorio de GitHub, vayan a **Settings** → **Secrets and variables** → **Actions** → **New repository variable**.
6. Nombre: `SONAR_ORG`. Valor: el organization key.

7. Por último en el repositorio de GitHub agregen la variable de Project Key, vayan a **Settings → Secrets and variables → Actions → New repository variable**.

   El valor de project key id se puede obtener de la URL del proyecto en Sonar.

   Ejemplo:

https://sonarcloud.io/project/information?id=josedelcastillo_INF384-lab2-20050452

8. Nombre: `SONAR_PROJECT_KEY`. Valor: (id de la URL)

   En el ejemplo sería “josedelcastillo_INF384-lab2-20050452”

C.5 — Línea base

1. Ejecuten el workflow **tres veces** desde la pestaña Actions, con **Run workflow**. No modifiquen ningún archivo.
2. Revisen las ejecuciones y asegúrense de tener 3 ejecuciones exitosas, si alguna falla ejecútalo nuevamente. Anota para cada ejecución en el archivo del repositorio “docs/linea-base.md”

Ejecución	Duración	URL
1		
2		
3		

3. Recuerda subir el archivo actualizado a tu repositorio.

Este número es el punto de partida. En la sesión van a comparar contra él, así que si no está, no tienen contra qué comparar.

C.6 — Registro

Registren la URL del repositorio en la tarea de PAIDEIA.

Restricciones

No modifiquen el workflow ni la configuración del repositorio antes de la sesión. La intervención se hace durante el laboratorio. Un repositorio que llegue ya modificado no sirve para el ejercicio.

El repositorio es público. No pongan credenciales, tokens ni datos personales en ningún archivo. El token de SonarCloud va únicamente como secreto del repositorio, nunca en el código.

Uso de IA generativa. Si la usan en cualquier parte de este trabajo previo, decláralo y cítenlo conforme al sílabo del curso.

Qué tienen que tener el día del laboratorio

- Su fork público, renombrado con su código PUCP y con **Actions habilitado**
- `docs/linea-base.md` con las tres ejecuciones y sus URLs
- El proyecto en SonarCloud, con Automatic Analysis desactivado
- El secreto `SONAR_TOKEN` y la variable `SONAR_ORG` configurado en el repositorio como secreto.
- La project key y la organization key anotadas